

Project 3 Overview

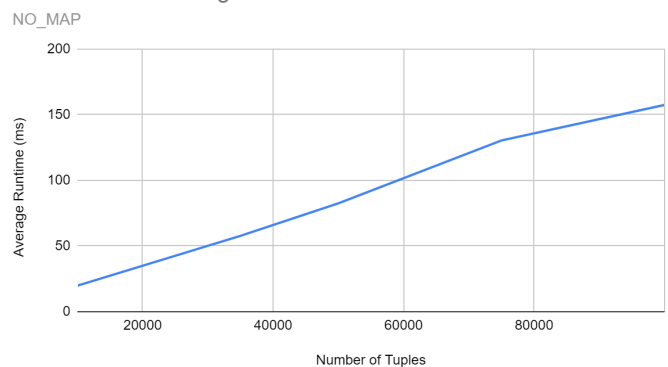
In this project our goal was to compare the performance of NO_MAP, TREE_MAP, HASH_MAP, and BPTREE_MAP for the select and join operations from our prior projects implementation. The performance was based on the number of tuples per millisecond using units of 10K tuples.

Performance Evaluation

- **NO_MAP by Armaity Katki**
 - Meaning that no map structure was used and the program doesn't use a specific map implementation to store key-value pairs.

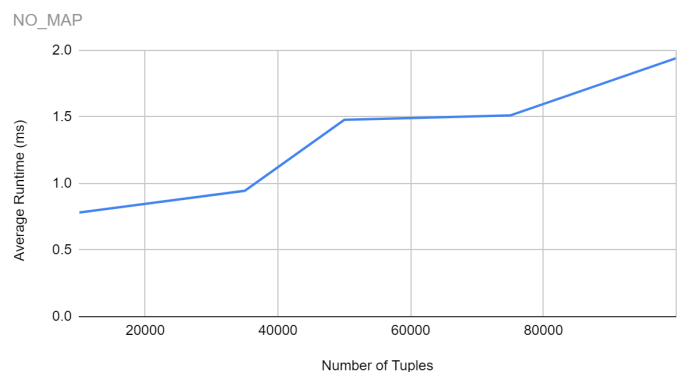
Select Case 1 - String	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	19.799
35000	57.717
50000	82.545
75000	130.420
100000	157.441

Select Case 1 String



Select Case 2 - Predicate	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	.782
35000	.945
50000	1.478
75000	1.511
100000	1.941

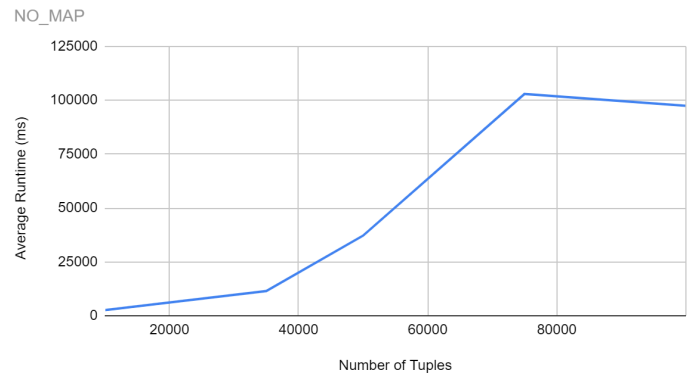
Select Case 2 - Predicate



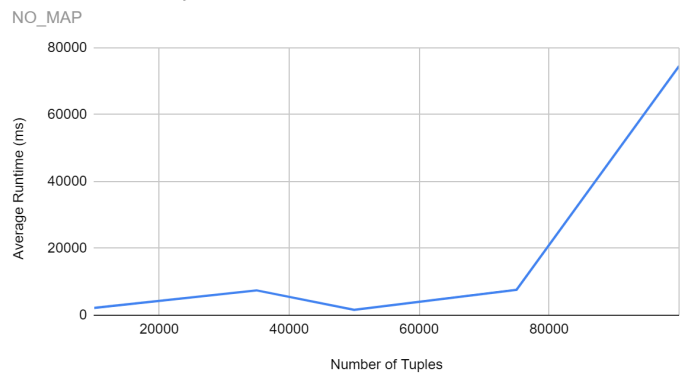
<u>Join Case 1 - Natural Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	2589.708
35000	11460.145
50000	37170.747
75000	102926.629
100000	97474.042

<u>Join Case 2 - Equi-Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	2208.639
35000	7432.265
50000	1609.341
75000	7600.849
100000	74519.399

Join Case 3 - Natural Join



Join Case 4 - Equi-Join

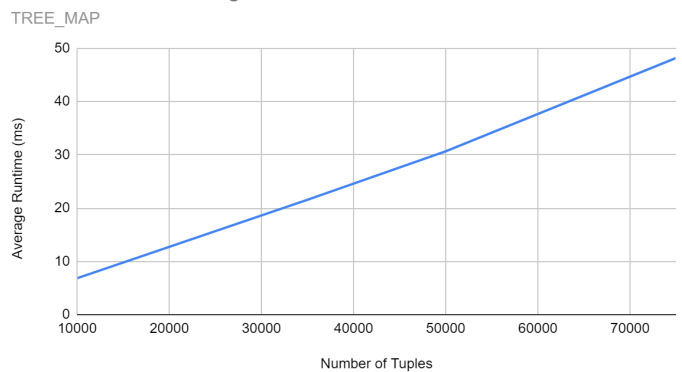


- **TREE_MAP by Ruthvik Mankari**

- This is implemented using a balanced tree structure which keeps the elements in sorted order based on their keys. This map operates in $O(\log(n))$ time.

<u>Select Case 1 - String</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	6.831
35000	21.568

Select Case 1 - String



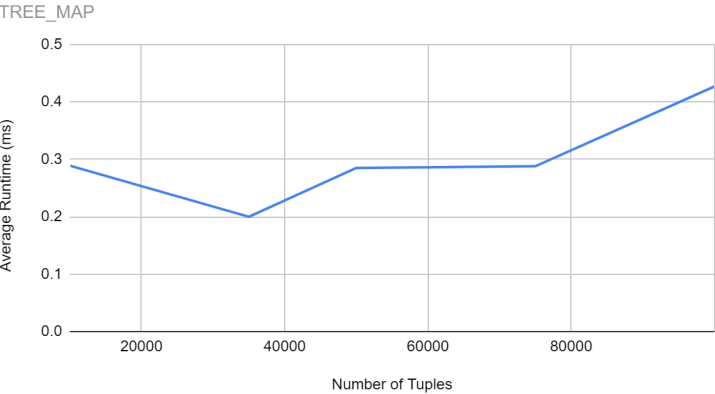
50000	30.660
75000	48.205
100000	66.697

Select Case 2 - Predicate	
Tuples	Average Runtime (milliseconds)
10000	.289
35000	.200
50000	.285
75000	.288
100000	.427

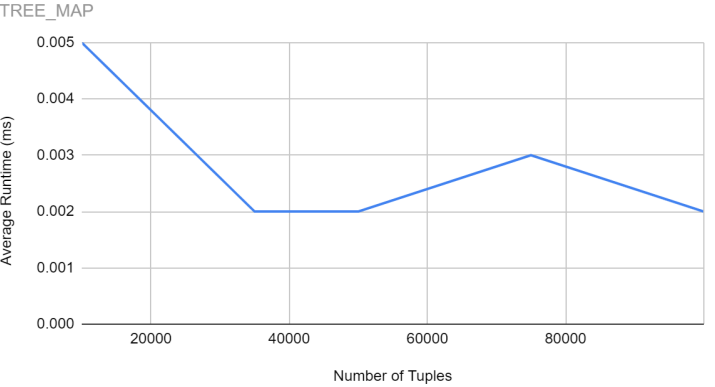
Select Case 3 - Indexed Select	
Tuples	Average Runtime (milliseconds)
10000	.005
35000	.002
50000	.002
75000	.003
100000	.002

Join Case 1 - Natural Join	
Tuples	Average Runtime (milliseconds)
10000	1207.891
35000	5344.661

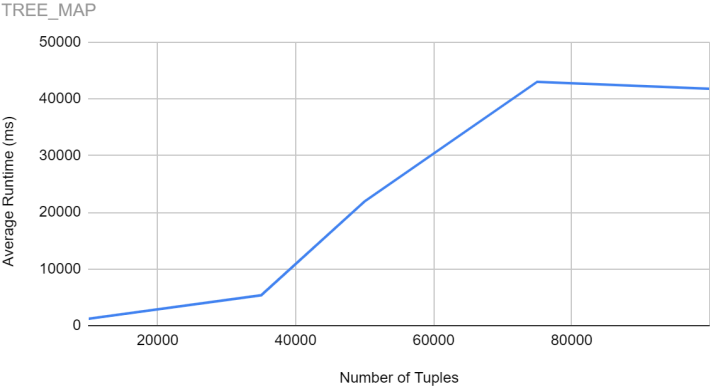
Select Case 2 - Predicate



Select Case 3 - Indexed Select



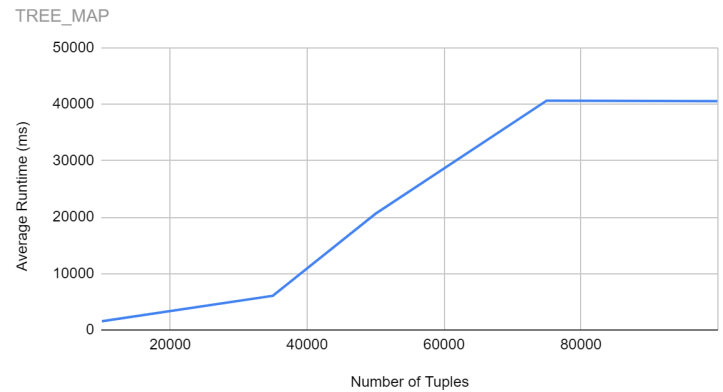
Join Case 1 - Natural Join



50000	21918.050
75000	43020.482
100000	41803.200

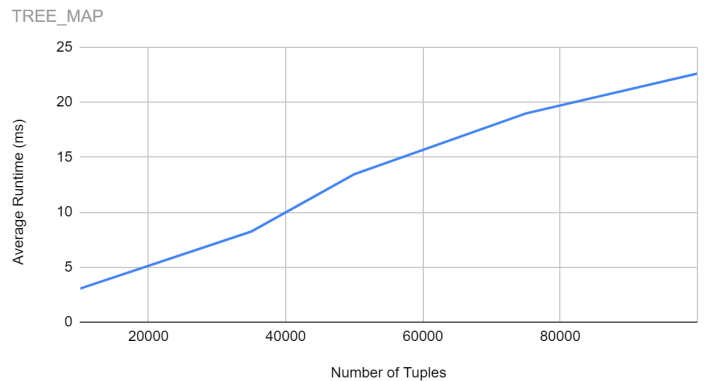
Join Case 2 - Equi-Join	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	1537.747
35000	6055.425
50000	20606.879
75000	40647.636
100000	40545.125

Join Case 2 - Equi-Join



Join Case 3 - Indexed Join	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	3.043
35000	8.242
50000	13.455
75000	18.981
100000	22.612

Join Case 3 - Indexed Join

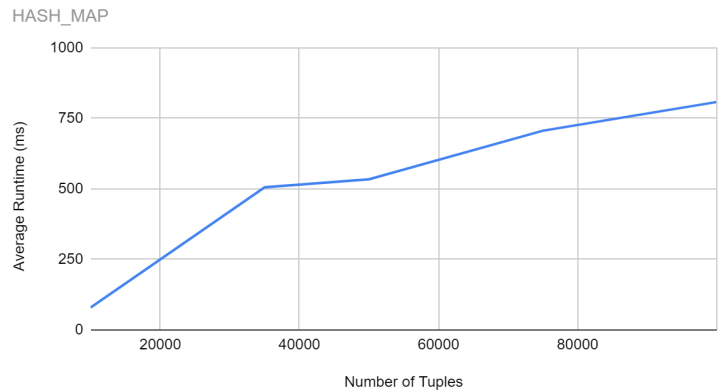


- **HASH_MAP by Nehaa Unapathy**

- This is implemented using a hash function to make keys to an index in an array. The average time complexity of this map is $O(1)$ and the worst case is $O(n)$.

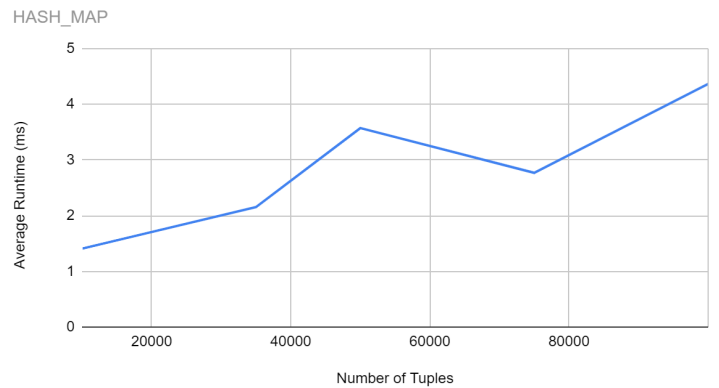
Select Case 1 - String	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	79.837
35000	505.637
50000	534.034
75000	706.300
100000	807.960

Select Case 1 - String



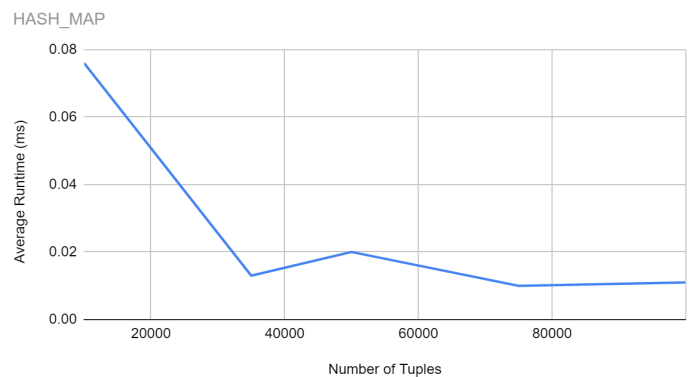
Select Case 2 - Predicate	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	1.408
35000	2.153
50000	3.573
75000	2.769
100000	4.365

Select Case 2 - Predicate



Select Case 3 - Indexed Select	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	.076
35000	.013
50000	.020
75000	.010
100000	.011

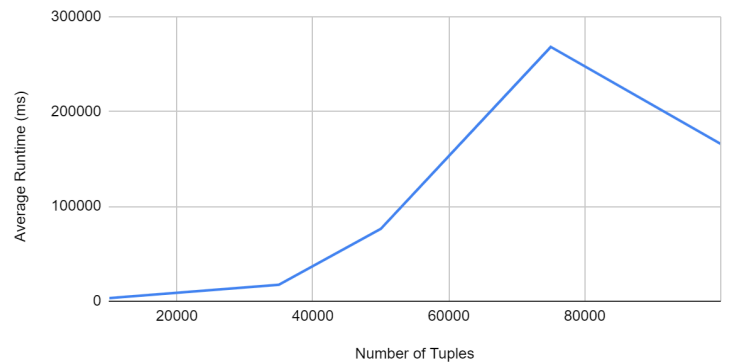
Select Case 3 - Indexed Select



<u>Join Case 1 - Natural Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	3238.187
35000	17341.196
50000	76365.794
75000	268135.361
100000	165784.939

Join Case 1 - Natural Join

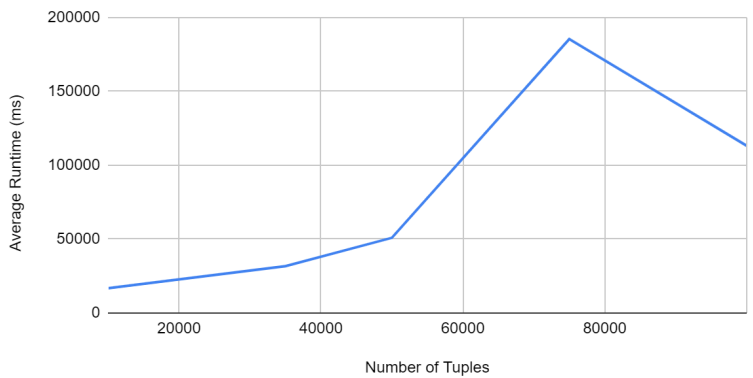
HASH_MAP



<u>Join Case 2 - Equi-Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	16755.765
35000	31727.574
50000	50902.980
75000	185435.858
100000	113210.613

Join Case 2 - Equi-Join

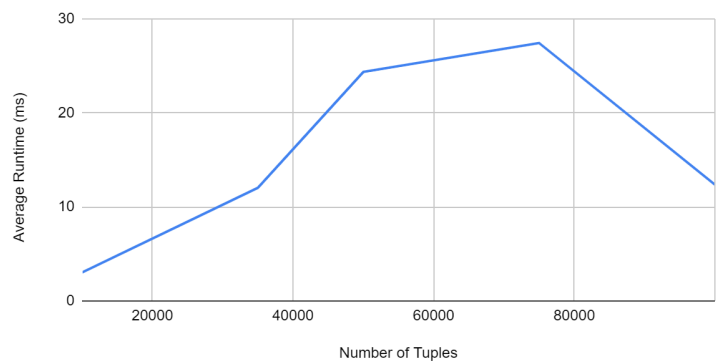
HASH_MAP



<u>Join Case 3 - Indexed Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	3.030
35000	12.033
50000	24.358

Join Case 3 - Indexed Join

HASH_MAP



75000	27.431
100000	12.367

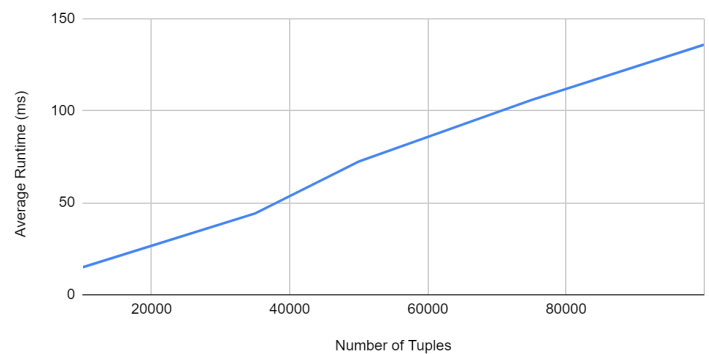
- **BPTREE_MAP by Luke Zimmerman**

- This is implemented using a B+ Tree structure, which is an optimized version of B-trees. B+ Trees are self balancing and maintain the data in a sorted order.

Select Case 1 - String	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	14.906
35000	44.247
50000	72.449
75000	105.864
100000	135.929

Select Case 1 - String

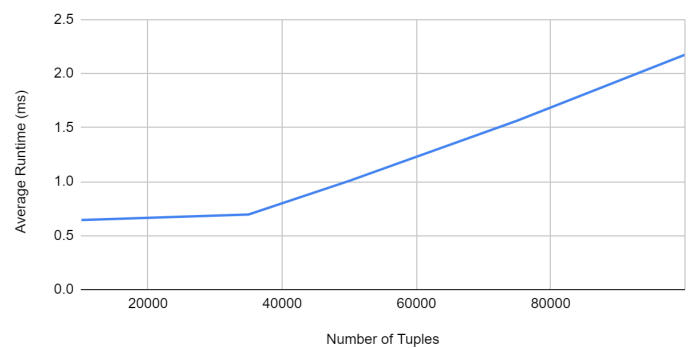
BPTREE_MAP



Select Case 2 - Predicate	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	.644
35000	.695
50000	1.007
75000	1.564
100000	2.175

Select Case 2 - Predicate

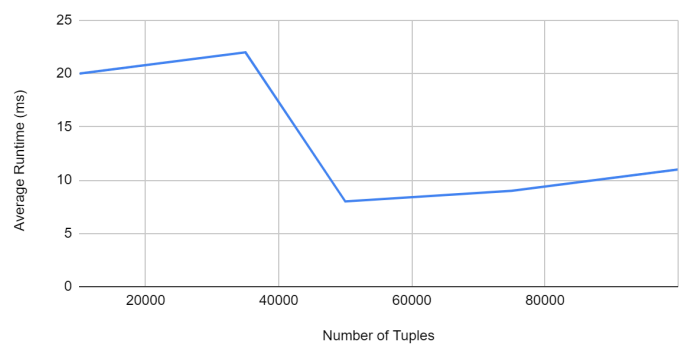
BPTREE_MAP



Select Case 3 - Indexed Select	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	20.0
35000	22.0
50000	8.0
75000	9.0
100000	11.0

Select Case 3 - Indexed Select

BPTREE_MAP

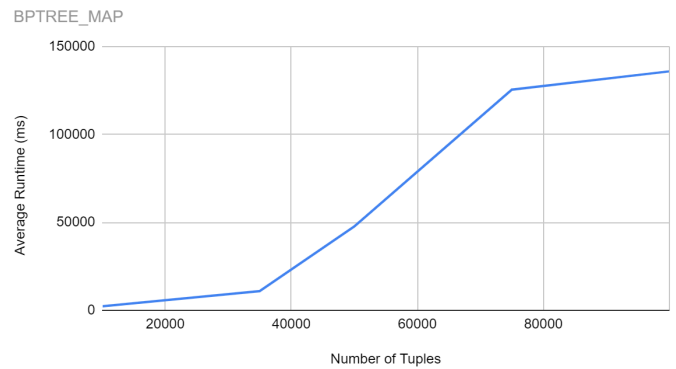


10000	20
35000	22
50000	8
75000	9
100000	11

* ran on a lowered number of tuples and scaled to a factor of 1000 at runtime.

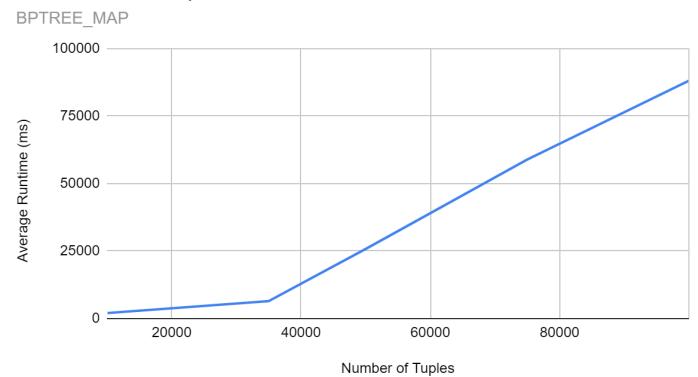
<u>Join Case 1 - Natural Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	2268.226
35000	10935.729
50000	47666.779
75000	125495.055
100000	135880.673

Join Case 1 - Natural Join



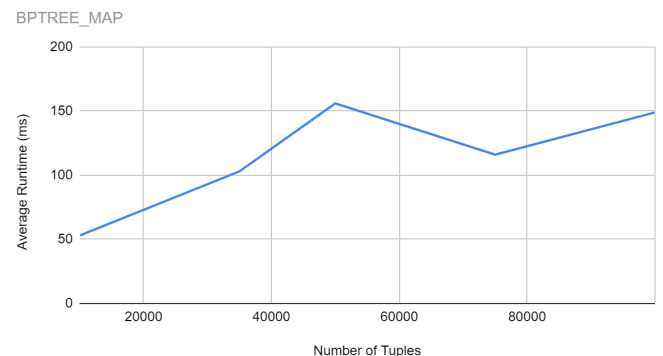
<u>Join Case 2 - Equi-Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	2010.462
35000	6432.814
50000	25777.653
75000	58899.439
100000	88095.279

Join Case 2 - Equi-Join



<u>Join Case 3 - Indexed Join</u>	
<i>Tuples</i>	<i>Average Runtime (milliseconds)</i>
10000	50
35000	100
50000	150
75000	120
100000	150

Join Case 3 - Indexed Join



10000	53
35000	103
50000	156
75000	116
100000	149

* ran on a lowered number of tuples and scaled to a factor of 1000 at runtime.

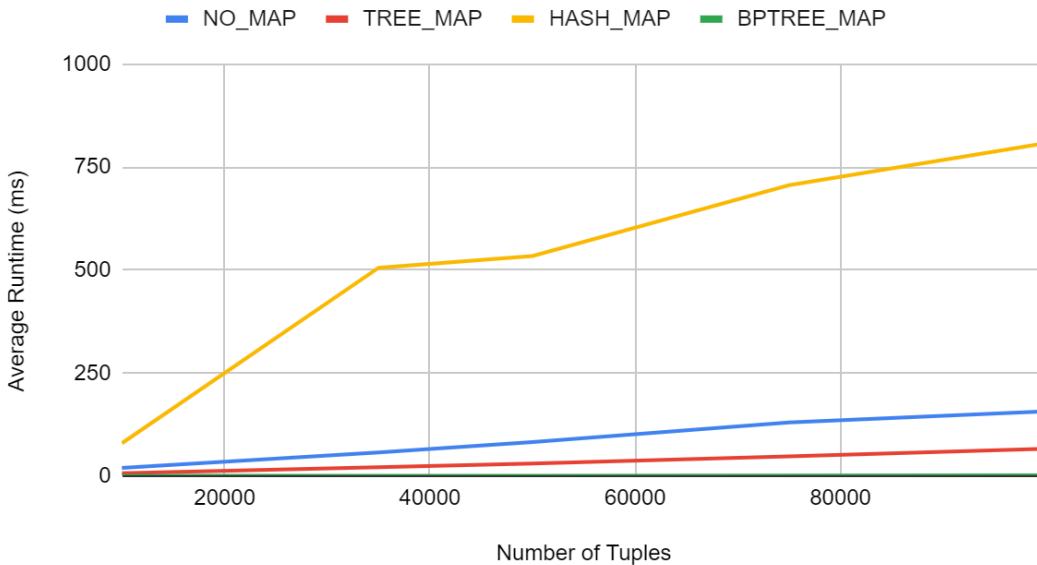
Performance Comparison

Now we will compare each mapping structure with each other in each case. The fastest will be bolded in each table.

- **Select Case 1 - String**

<i>Number of Tuples</i>	<i>NO_MAP</i>	<i>TREE_MAP</i>	<i>HASH_MAP</i>	<i>BPTREE_MAP</i>
10000	19.799	6.831	79.837	0.644
35000	57.717	21.568	505.637	0.695
50000	82.545	30.66	534.034	1.007
75000	130.42	48.205	706.3	1.564
100000	157.441	66.697	807.96	2.175

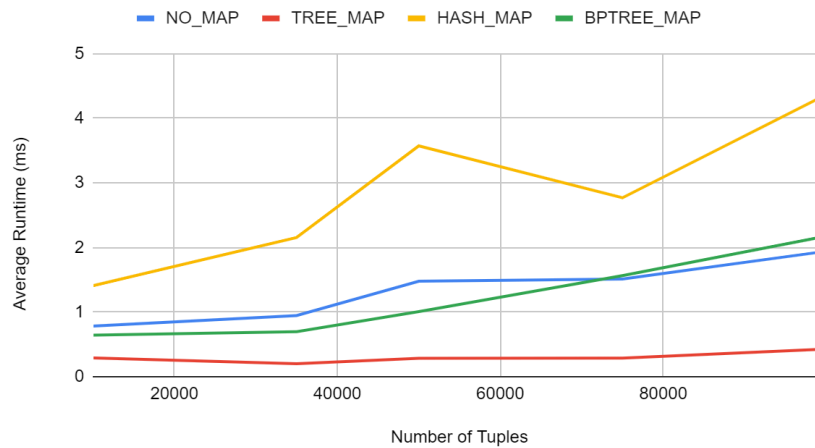
Select Case 1 - String



- **Select Case 2 - Predicate**

<i>Number of Tuples</i>	<i>NO_MAP</i>	<i>TREE_MAP</i>	<i>HASH_MAP</i>	<i>BPTREE_MAP</i>
10000	0.782	0.289	1.408	0.644
35000	0.945	0.2	2.153	0.695
50000	1.478	0.285	3.573	1.007
75000	1.511	0.288	2.769	1.564
100000	1.941	0.427	4.365	2.175

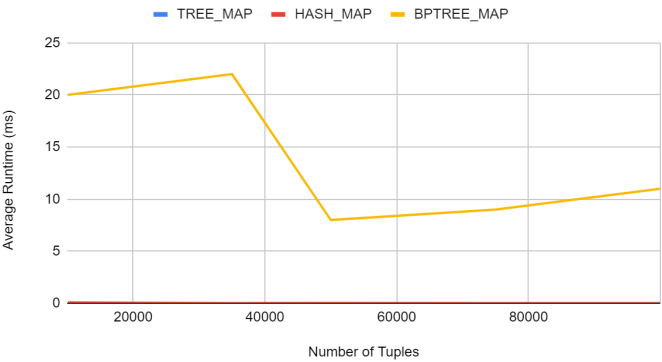
Select Case 2 - Predicate



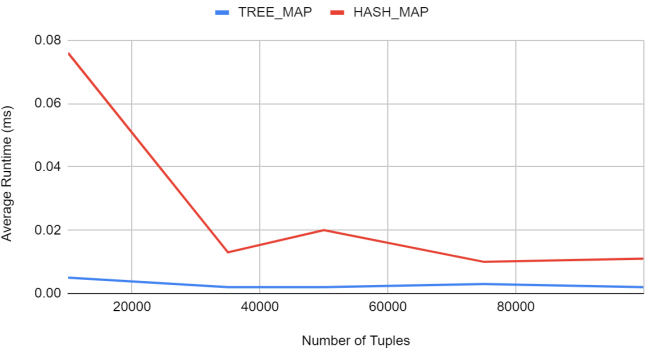
- **Select Case 3 - Indexed Select (excluding NO_MAP)**

<i>Number of Tuples</i>	<i>TREE_MAP</i>	<i>HASH_MAP</i>	<i>BPTREE_MAP</i>
10000	0.005	0.076	20
35000	0.002	0.013	22
50000	0.002	0.02	8
75000	0.003	0.01	9
100000	0.002	0.011	11

Select Case 3 - Indexed Select



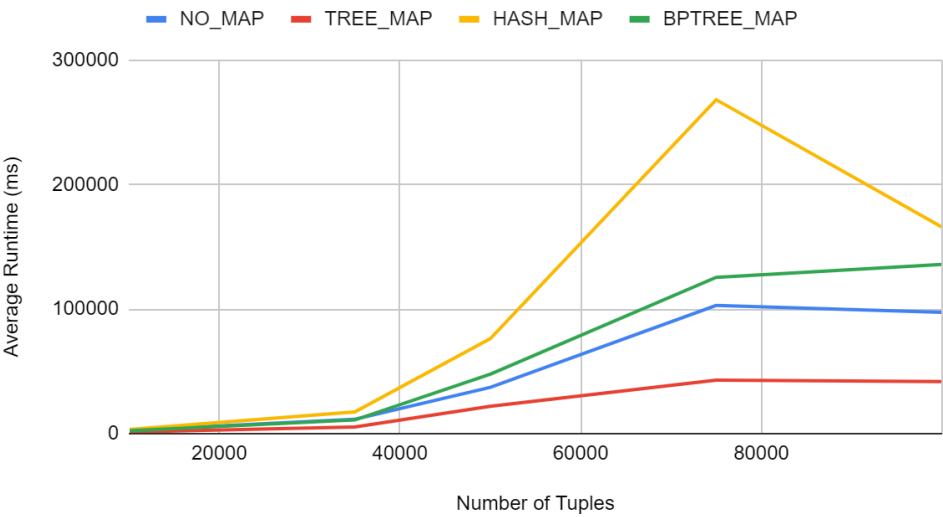
Select Case 3 - Indexed Select



• Join Case 1 - Natural Join

Number of Tuples	NO_MAP	TREE_MAP	HASH_MAP	BPTREE_MAP
10000	2589.708	1207.891	3238.187	2268.226
35000	11460.145	5344.661	17341.196	10935.729
50000	37170.747	21918.05	76365.794	47666.779
75000	102926.629	43020.482	268135.361	125495.055
100000	97474.042	41803.2	165784.939	135880.673

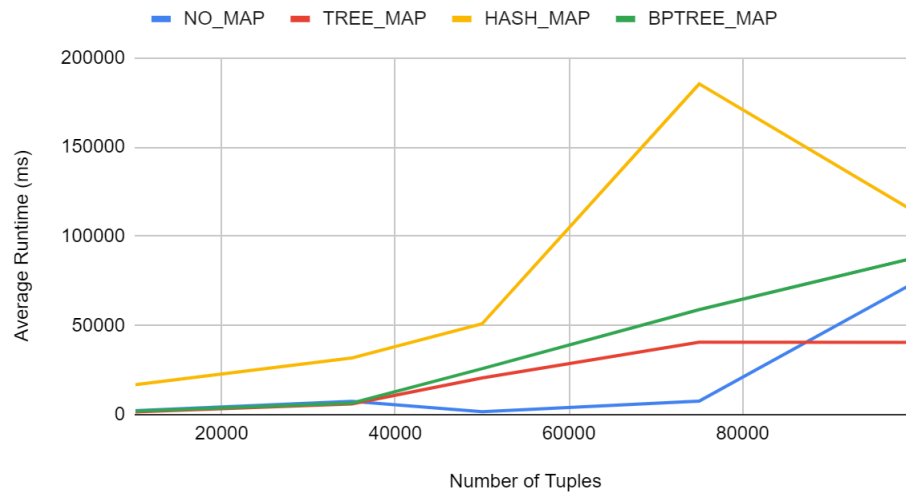
Join Case 1 - Natural Join



- **Join Case 2 - Equi-Join**

<i>Number of Tuples</i>	<i>NO_MAP</i>	<i>TREE_MAP</i>	<i>HASH_MAP</i>	<i>BPTREE_MAP</i>
10000	2208.639	1537.747	16755.765	2010.462
35000	7432.265	6055.425	31727.574	6432.814
50000	1609.341	20606.879	50902.98	25777.653
75000	7600.849	40647.636	185435.858	58899.439
100000	74519.399	40545.125	113210.613	88095.279

Join Case 2 - Equi-Join



- **Join Case 3 - Index Join (Excluding NO_MAP)**

<i>Number of Tuples</i>	<i>TREE_MAP</i>	<i>HASH_MAP</i>	<i>BPTREE_MAP</i>
10000	3.043	3.03	53
35000	8.242	12.033	103
50000	13.455	24.358	156
75000	18.981	27.431	116
100000	22.612	12.367	149

*TREE_MAP italicized due to it being a close second.

Join Case 3 - Indexed Join

